

# The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch06\_classification

AYuSh

# Contents

|                                           |   |
|-------------------------------------------|---|
| Chapter 6: Classification Algorithms..... | 3 |
|-------------------------------------------|---|

# Chapter 6: Classification Algorithms

---

The four algorithms in this chapter — K-Nearest Neighbors, Naive Bayes, Support Vector Machines, and Decision Trees — are interview fixtures for a reason that has little to do with production usage numbers: each one isolates a fundamental idea in its purest form. KNN is pure memory: no training, no parameters, just distances — the cleanest possible probe of whether you understand bias-variance, the curse of dimensionality, and why scaling matters. Naive Bayes is Bayes' theorem turned into a classifier by one heroic assumption, and the discussion of when that assumption's wrongness matters is a probability interview disguised as an ML one. SVMs carry the margin story and the kernel trick — the most elegant idea in classical ML, and a perennial "explain it precisely" request. Decision trees are greedy recursive partitioning with information theory as the split judge, and they are the atoms from which random forests and gradient boosting (Chapter 7) are built — you cannot understand the ensembles that dominate tabular ML without understanding the tree's specific failure mode they were designed to fix.

For each algorithm this chapter follows the same arc: the mechanism from zero, the mathematics with every symbol defined, what the hyperparameters really control (always, in the end, bias versus variance — Chapter 4's dial wearing four costumes), where the algorithm breaks, and what interviewers probe. Logistic regression, the fifth member of the classical-classifier canon, has its own treatment in Chapter 5.

## K-Nearest Neighbors

KNN classifies a point by plurality vote of its  $k$  closest training examples. That single sentence is the entire algorithm: there is no training phase beyond storing the data, no parameters are fit, and all computation is deferred to prediction time — the textbook example of a **lazy** (instance-based), **non-parametric** learner (Chapter 4's taxonomy). Its inductive bias is bare smoothness: points near each other probably share a label, and nothing more.

Formally: given query  $x$ , distance metric  $d(\cdot, \cdot)$ , and the set  $N_k(x)$  of the  $k$  training points closest to  $x$ , predict

$$\hat{y}(x) = \operatorname{argmax}_c \sum_{i \in N_k(x)} \mathbf{1}(y_i = c)$$

— optionally weighting each vote by  $1/d(x, x_i)$  so nearer neighbors count more (this also breaks ties gracefully and softens the choice of  $k$ ). Probability estimates fall out as vote fractions, though they are coarse for small  $k$ . The regression variant averages neighbor targets instead of voting; everything below applies to it unchanged.

**k is the bias-variance dial, exactly.**  $k = 1$  memorizes: training accuracy is 1.0 by construction (every point is its own nearest neighbor) and the decision boundary is a jagged Voronoi mosaic that traces every noise point — maximal variance.  $k = n$  predicts the global majority class everywhere — maximal bias. In between, larger  $k$  averages over more neighbors, smoothing the boundary: variance falls, bias rises. Listing 1 walks the dial on two-moons data:  $k=1$  gives 1.000 train / 0.892 test (the overfit gap),  $k=15$  the best test accuracy at 0.896, and  $k=201$  sags to 0.821 as the vote pools the two moons together. Choose  $k$  by cross-validation; odd  $k$  avoids ties in binary problems; and a rule-of-thumb starting point is  $k \approx \sqrt{n}$ , said with the caveat that CV decides.

**Distance is the model.** Euclidean ( $\ell_2$ ) is the default; Manhattan ( $\ell_1$ ) is less dominated by any single coordinate; cosine similarity serves high-dimensional sparse data like text (Chapter 19); Hamming handles categorical vectors. Because raw distances add feature contributions in squared units, **KNN is defenseless against scale**: a feature measured in grams contributes a million times the squared difference of the same feature in kilograms, silently deciding every neighborhood by itself. Listing 1 stages this — multiplying one coordinate by 1,000 drops accuracy from 0.887 to 0.792, and standardization restores it. Scale features, always, before any distance-based method (the same imperative as Chapter 5's gradient descent and regularization, for a different reason: there it was the optimizer and the penalty; here the scale-sensitivity *is the model*).

**The curse of dimensionality bites KNN first and hardest.** Chapter 4 established the geometry: in high dimensions, distances concentrate — the nearest and farthest neighbors become nearly equidistant, and "nearest" stops carrying information. KNN's entire signal is the distance ranking, so it degrades before almost anything else. Practical mitigations: dimensionality reduction first (PCA, Chapter 8; learned embeddings, Chapters 12+), feature selection (Chapter 9), or metric learning. The modern redemption arc is worth one interview sentence: KNN over *learned embeddings* is exactly how retrieval systems, vector databases, and RAG (Chapter 24) work — the deep network fixes the representation so that distance means something, then KNN does what it always did, at billion-point scale via approximate indexes (HNSW, IVF — Chapter 24).

**Cost profile.** Training  $O(1)$  beyond storage; prediction  $O(nd)$  per query, brute force — backwards from most learners, and disqualifying for latency-critical serving at scale unless you accept approximate neighbors (KD-trees and ball trees help only in low dimension; past a few dozen dimensions they degenerate to linear scans). Memory: the entire training set, forever. Imbalance: the majority class wins votes just by density — distance weighting and stratified sampling help. KNN's honest niche: small-to-medium tabular data, strong local structure, zero training budget, and as the algorithmic core of embedding retrieval.

## Naive Bayes

Naive Bayes classifies with Bayes' theorem (Chapter 1), choosing the class with the highest posterior:

$$p(y = c \mid x) \propto p(y = c) \prod_{j=1}^d p(x_j \mid y = c)$$

The product is the "naive" step: it assumes features are **conditionally independent given the class** — knowing the class, seeing one feature tells you nothing further about another. This is false essentially always ("free" and "cash" co-occur in spam far beyond independence), and the algorithm works anyway; resolving that tension is the standard interview probe, and the resolution is precise: classification needs only the *argmax* over classes, not correct probabilities. Double-counting correlated evidence inflates the winning posterior's confidence, but often leaves the *ranking* of classes intact — decisions survive, calibration dies. So: use the labels, distrust the probabilities (recalibrate — Platt scaling or isotonic regression, Chapter 10 — if you need them).

Training is one counting pass: estimate the prior  $p(y = c)$  as class frequency, and each  $p(x_j \mid y = c)$  from the class's data alone — a per-feature, per-class estimate that never touches feature interactions. That is why NB is fast (one pass,  $O(nd)$ ), why it tolerates high dimension (it never estimates a joint), and why it works from shockingly little data ( $d$  univariate estimates need far fewer samples than one  $d$ -variate estimate — a bias-variance argument: NB is a high-bias, low-variance model that wins exactly when data is scarce relative to dimension).

The three variants differ only in the likelihood model for  $p(x_j \mid y = c)$ , keyed to feature type:

- **Gaussian NB** — continuous features: fit a per-class mean and variance per feature, evaluate the normal density. Listing 2 implements it from scratch in ~20 lines (means, variances, log-densities, *argmax*) and matches *sklearn* to the fourth decimal.
- **Multinomial NB** — count features, the text-classification workhorse:  $p(w \mid c)$  is the smoothed fraction of class- $c$  word occurrences that are word  $w$ ; a document's log-score is the count-weighted sum of word log-probabilities plus the log prior.
- **Bernoulli NB** — binary presence/absence features: models each feature as a coin flip per class, and — the distinction interviewers fish for — **explicitly penalizes absent words** (the  $(1 - p)$  factors), where Multinomial simply ignores them. Bernoulli can beat Multinomial on short documents where absence is informative; Multinomial wins on longer text where counts carry signal.

**Numerics and smoothing are where implementations fail.** A product of hundreds of small probabilities underflows to zero — always work in log space, turning products into sums (Listing 3 does, as does every serious implementation). And an unseen word-class

pair gives  $p = 0$ , whose log is  $-\infty$ , vetoing the entire class on one missing count.

**Laplace (add-one) smoothing** fixes it:

$$\hat{p}(w | c) = \frac{\text{count}(w, c) + \alpha}{\sum_{w'} \text{count}(w', c) + \alpha V}$$

with  $\alpha = 1$  classic,  $\alpha$  tunable, and  $V$  the vocabulary size — a Dirichlet-prior MAP estimate in disguise (Chapter 1 again). Listing 3 builds the whole spam filter from scratch — vectorize, count, smooth, sum logs — agrees with sklearn, and shows the un-smoothed zero that would have killed it.

Where NB earns its keep in 2026: text baselines that take seconds and embarrass slower models more often than anyone admits, spam filtering, streaming/online settings (counts update incrementally), and any tiny-data regime. Its known lies: correlated features double-counted (two copies of the same feature literally squares its evidence), and probabilities pushed toward 0/1 — overconfident by construction.

## Support Vector Machines

A linear classifier's decision boundary is a hyperplane  $w^\top x + b = 0$ ; among all hyperplanes that separate the classes, which one? The perceptron takes any; logistic regression takes the one maximizing likelihood. The SVM's answer: the one that **maximizes the margin** — the distance to the nearest training point of either class. Wide margins are a variance-control argument before they are anything else: a boundary with clearance survives perturbations of the data that a grazing boundary does not, and the formal statement (margin-based generalization bounds independent of dimension) is why SVMs dominated the 2000s.

**Hard margin.** Scale  $w, b$  so the closest points satisfy  $|w^\top x + b| = 1$  (the canonical form). The margin — distance between the two supporting hyperplanes  $w^\top x + b = \pm 1$  — is  $2/\|w\|$ , so maximizing margin is minimizing  $\|w\|^2$  subject to every point classified with clearance:

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{s. t.} \quad y_i(w^\top x_i + b) \geq 1 \quad \forall i$$

with labels  $y_i \in \{-1, +1\}$ . A convex quadratic program: unique solution, no local minima. The points that achieve equality — the ones touching the margin — are the **support vectors**; the solution is a weighted combination of them alone, and deleting every other training point changes nothing. That sparsity is the SVM's signature fact.

**Soft margin.** Real data is not separable, and hard margins are hostage to their single closest point. Introduce slack  $\xi_i \geq 0$  measuring each point's violation and charge for it:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \quad \text{s. t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i$$

**C is the bias-variance dial:** large C punishes violations severely — narrow margin, boundary contorted around individual points, high variance; small C tolerates violations for width — smoother boundary, higher bias. (Note the inversion relative to  $\lambda$  in Chapter 5: C multiplies the *loss*, not the penalty, so  $C \approx 1/\lambda$  — the same gotcha as sklearn's LogisticRegression.) Listing 4 makes it tangible: C from 0.01 to 100 shrinks the geometric margin from 3.88 to 1.04 and the support-vector count from 48 to 7. Equivalent unconstrained view for the loss-function fluent: soft-margin SVM minimizes **hinge loss**  $\max(0, 1 - y_i(w^\top x_i + b))$  plus L2 — zero loss for points beyond the margin (hence sparsity and robustness to easy points), linear loss for violators (hence robustness to outliers relative to squared losses); compare cross-entropy, which never reaches exactly zero and keeps pulling on well-classified points.

**The kernel trick.** In the dual formulation of the QP, training data enters *only through inner products*  $x_i^\top x_j$ , and prediction only through  $x^\top x_i$  against support vectors. So replace every inner product with a **kernel**  $K(a, b) = \phi(a)^\top \phi(b)$  — the inner product in some feature space  $\phi$  maps to — and you have fit a maximum-margin hyperplane in that space *without ever computing*  $\phi$ . Listing 5 shows the books balancing exactly: for the degree-2 kernel  $K(a, b) = (a^\top b)^2$  in 2-D, the explicit map is  $\phi(a) = (a_1^2, a_2^2, \sqrt{2}a_1a_2)$ , and  $(a^\top b)^2 = \phi(a)^\top \phi(b)$  to machine precision. The economics: the RBF kernel  $K(a, b) = \exp(-\gamma\|a - b\|^2)$  corresponds to an *infinite-dimensional* feature space, computed in  $O(d)$  per pair. A hyperplane in that space is a smooth nonlinear surface in the original one — Listing 5's concentric circles go from 0.510 test accuracy (linear: a hyperplane cannot enclose a disk) to 0.995 (poly or RBF). Validity: any symmetric positive semi-definite function is a legal kernel (Mercer's condition); sums and products of kernels are kernels.

**Kernel choice and  $\gamma$ .** Linear for high-dimensional sparse data (text — often already separable; use LinearSVC/liblinear, which scales far better); RBF as the default nonlinear choice; polynomial when interaction structure of known degree is plausible. RBF's  $\gamma$  is a *second* variance dial: it sets the kernel's reach ( $\gamma = 1/(2\sigma^2)$  in bandwidth terms), so large  $\gamma$  means each support vector influences only its immediate vicinity — boundaries shrink-wrap the data, overfitting; small  $\gamma$  approaches a linear-ish smooth boundary. C and  $\gamma$  interact; tune them jointly on a log grid. And scale features first — the RBF kernel is a distance computation, inheriting KNN's scale-sensitivity verbatim.

Practical profile. Training is between  $O(n^2)$  and  $O(n^3)$  for kernel SVMs — past  $\sim 100k$  points, use linear SVMs or different models entirely (this scaling, plus deep learning eating the perception domains and boosting eating tabular, explains the SVM's fall from default status). No native probabilities (Platt scaling bolts them on, at the cost of a held-out fit); multiclass via one-vs-rest or one-vs-one (libsvm's choice); memory is the support-vector set at prediction time. Their enduring niches: medium-sized clean datasets, high-dimensional/low-sample regimes (genomics), and interviews — because margin + duality + kernels remains the densest concentration of beautiful, testable theory in classical ML.

## Decision Trees

A decision tree classifies by asking a sequence of feature questions — is income  $\leq 55k$ ? is default\_history = 1? — routing each example down branches to a leaf, and predicting the leaf's majority class (or class proportions). It is the only model in this chapter whose decision process a non-technical stakeholder can read directly, and the only one indifferent to feature scaling and monotone transforms (splits compare against thresholds; only the *order* of values matters — log-transforming a feature changes nothing).

**Growing the tree is greedy recursive partitioning.** At each node, try every feature and every threshold; score each candidate split by how much it purifies the labels; take the best; recurse on the two children; stop when leaves are pure or a limit is hit. Exhaustive search over tree structures is NP-hard; greedy top-down is the universal compromise, and its myopia is real — a split useless now may enable a great one later (XOR is the classic construction: neither feature alone reduces impurity, so a greedy tree can't start), which foreshadows why ensembles of trees beat single trees.

**Impurity measures** make "purifies" precise. For a node with class proportions  $p_c$ :

$$\text{Entropy} = -\sum_c p_c \log_2 p_c \quad \text{Gini} = 1 - \sum_c p_c^2$$

Both are zero for a pure node and maximal for a uniform mix (entropy 1 bit, Gini 0.5 for balanced binary). Gini reads as "probability of misclassifying a random node member if labeled by the node's own class distribution"; entropy is Chapter 1's information measure. **Information gain** is the impurity drop from a split — parent impurity minus the size-weighted average of child impurities:

$$\text{Gain} = I(\text{parent}) - \frac{n_L}{n} I(\text{left}) - \frac{n_R}{n} I(\text{right})$$

Listing 6 computes everything by hand on a 12-row loan dataset: parent entropy exactly 1 bit (6/6 split), and both criteria select the same split (default\_history) with gains of 0.655 bits and 0.357 Gini. **Gini vs entropy in practice:** they disagree rarely and unsystematically — Listing 8 measures it across 40 datasets: gini "wins" 22, entropy 18, mean absolute accuracy difference 0.019, identical fit times. Gini is the sklearn default (no logarithm to compute); the honest interview answer is that the criterion is a rounding error next to depth control. One real distinction worth knowing: information gain is biased toward high-cardinality features (an ID column splits into pure singletons for spectacular fake gain), which C4.5 corrects with **gain ratio** (gain normalized by the split's own entropy); continuous features are handled by sorting values and testing boundary thresholds, categorical ones by subset or one-hot splits depending on the implementation.

**Trees overfit by default, and pruning is the cure.** Grown to purity, a tree memorizes — one leaf per hard example, boundaries tracing noise. Listing 7, with 15% label noise: the full tree has 90 leaves, train accuracy 1.000, test 0.798; depth-capped versions do



better; and **cost-complexity pruning** — grow full, then collapse subtrees whose accuracy contribution doesn't justify their leaves, choosing the penalty  $\alpha$  by cross-validation — lands at 11 leaves and test 0.842, beating every fixed depth. Pre-pruning alternatives (`max_depth`, `min_samples_leaf`, `min_impurity_decrease`) are cheaper and tunable the same way; post-pruning sees the whole tree before deciding, dodging the greedy horizon problem. Either way, the tree's variance problem is structural: deep trees are wildly unstable — resample the data and the top split flips, cascading into a different tree entirely. That instability is precisely the property bagging exploits (Chapter 7: average many high-variance trees into one low-variance forest), which is why single trees are today a teaching tool and an ensemble ingredient more than a production model.

**Boundary geometry and blind spots.** Every split is axis-aligned, so tree boundaries are staircases; a diagonal linear boundary costs a deep staircase of splits that a linear model gets for free — worth naming as the dual of the SVM's weakness on rectangle-shaped truths. Trees extrapolate as constants (a leaf's prediction holds however far out you go), handle missing values via surrogate splits (CART) or learned default directions (XGBoost, Chapter 7), and their "feature importance" (total impurity reduction per feature) inherits the cardinality bias — permutation importance (Chapter 11) is the more honest read.

## Code implementations

Every listing was executed as shown; outputs are real. The set covers the four algorithms' defining behaviors: KNN's  $k$ -dial and scale fragility, Gaussian NB from scratch matching sklearn, a complete smoothed spam filter, the SVM margin shrinking as  $C$  grows, the kernel identity verified to machine precision, tree splits computed by hand, pruning beating every fixed depth, and the Gini-vs-entropy question settled empirically.

### Listing 1 — KNN: the $k$ dial and the scaling trap

$k=1$  posts perfect training accuracy by construction — each point is its own nearest neighbor — with the overfit gap to prove it.  $k=201$  pools the two moons into mush. The second half multiplies one feature by 1,000: neighborhoods are then decided by that feature alone, and accuracy drops nine points until standardization restores it.

```

"""Listing 1: KNN -- the k dial is a bias-variance dial, and scaling is not
optional."""
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
X, y = make_moons(n_samples=600, noise=0.3, random_state=0)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.4, random_state=0)

print(f"{'k':>4} {'train acc':>10} {'test acc':>9}")
for k in [1, 5, 15, 51, 201]:
    m = KNeighborsClassifier(n_neighbors=k).fit(X_tr, y_tr)
    print(f"{'k':>4} {m.score(X_tr, y_tr):>10.3f} {m.score(X_te, y_te):>9.3f}")

```

```
# Scaling: put one feature in "grams" (x1000) and watch distances break
X2 = X.copy(); X2[:, 1] *= 1000
X2_tr, X2_te, y2_tr, y2_te = train_test_split(X2, y, test_size=0.4, random_state=0)
raw = KNeighborsClassifier(15).fit(X2_tr, y2_tr).score(X2_te, y2_te)
sc = StandardScaler().fit(X2_tr)
scaled = KNeighborsClassifier(15).fit(sc.transform(X2_tr),
y2_tr).score(sc.transform(X2_te), y2_te)
print(f"\nfeature 2 in x1000 units: raw acc={raw:.3f} scaled acc={scaled:.3f}")
```

Output:

| k   | train acc | test acc |
|-----|-----------|----------|
| 1   | 1.000     | 0.892    |
| 5   | 0.939     | 0.887    |
| 15  | 0.917     | 0.896    |
| 51  | 0.914     | 0.879    |
| 201 | 0.825     | 0.821    |

```
feature 2 in x1000 units: raw acc=0.792 scaled acc=0.887
```

## Listing 2 — Gaussian Naive Bayes from scratch

The whole classifier is twenty lines: per-class priors, per-feature means and variances, log-densities summed under the independence assumption, argmax. Exact agreement with sklearn. Note the log-space computation and the variance floor ( $1e-9$ ) — the two numerics habits that keep NB implementations alive.

```
"""Listing 2: Gaussian Naive Bayes from scratch -- Bayes' rule plus one big
assumption."""
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

X, y = make_classification(n_samples=1500, n_features=6, n_informative=4,
                           n_redundant=0, random_state=4)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=4)

class ScratchGNB:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.prior, self.mu, self.var = {}, {}, {}
        for c in self.classes:
            Xc = X[y == c]
            self.prior[c] = len(Xc) / len(X)           # P(y=c)
            self.mu[c] = Xc.mean(axis=0)               # per-feature mean
            self.var[c] = Xc.var(axis=0) + 1e-9         # per-feature variance
        return self

    def _log_joint(self, X, c):
        # log P(y=c) + sum_j log N(x_j; mu_cj, var_cj) -- the naive independence step
        ll = -0.5 * (np.log(2 * np.pi * self.var[c]) + (X - self.mu[c])**2 /
self.var[c])
        return np.log(self.prior[c]) + ll.sum(axis=1)

    def predict(self, X):
        joint = np.column_stack([self._log_joint(X, c) for c in self.classes])
        return self.classes[joint.argmax(axis=1)]

acc_scratch = (ScratchGNB().fit(X_tr, y_tr).predict(X_te) == y_te).mean()
```

```
acc_sklearn = GaussianNB().fit(X_tr, y_tr).score(X_te, y_te)
print(f"scratch GNB: {acc_scratch:.4f}    sklearn GNB: {acc_sklearn:.4f}")
```

Output:

```
scratch GNB: 0.8022    sklearn GNB: 0.8022
```

### Listing 3 — Multinomial NB spam filter with Laplace smoothing

A complete text classifier from scratch: count vectorization, smoothed per-class word probabilities, log-space scoring, and agreement with sklearn. The last line shows the bug smoothing prevents: "deadline" never appears in spam, so its unsmoothed probability is zero, and one  $\log(0)$  would veto the spam class for any document containing it.

```
"""Listing 3: Multinomial NB for text -- word counts, log-space, Laplace smoothing."""
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

train_docs = ["win cash prize now", "claim your free prize", "win free cash now",
              "meeting at noon tomorrow", "project deadline next week",
              "lunch meeting with the team"]
train_y = np.array([1, 1, 1, 0, 0, 0]) # 1 = spam
test_docs = ["free cash prize meeting", "team project lunch tomorrow"]

vec = CountVectorizer()
Xtr = vec.fit_transform(train_docs).toarray()
Xte = vec.transform(test_docs).toarray()
vocab = np.array(vec.get_feature_names_out())
V = len(vocab)

# From scratch, with Laplace smoothing (alpha=1)
logp_w = {}
for c in [0, 1]:
    counts = Xtr[train_y == c].sum(axis=0)
    logp_w[c] = np.log((counts + 1) / (counts.sum() + V)) # P(word|class), smoothed
log_prior = {c: np.log((train_y == c).mean()) for c in [0, 1]}

for doc, x in zip(test_docs, Xte):
    scores = {c: log_prior[c] + (x * logp_w[c]).sum() for c in [0, 1]}
    pred = max(scores, key=scores.get)
    print(f"{doc!r}: spam-score={scores[1]:.2f} ham-score={scores[0]:.2f} -> {'SPAM'
    if pred else 'ham'}")

sk = MultinomialNB(alpha=1.0).fit(Xtr, train_y)
print("sklearn agrees:", sk.predict(Xte))
```

Output:

```
'free cash prize meeting': spam-score=-11.13 ham-score=-13.46 -> SPAM
'team project lunch tomorrow': spam-score=-14.43 ham-score=-11.78 -> ham
sklearn agrees: [1 0]
```

## Listing 4 — Soft-margin SVM: what C actually does

Same data, three values of C. The geometric margin ( $2/\|w\|$ ) shrinks from 3.88 to 1.04 as C rises, and the support-vector count falls from 48 to 7 — a wide tolerant boundary leaning on many points versus a narrow strict one leaning on few. Test accuracy barely moves here because the blobs are nearly separable; on noisier data, large C is where overfitting lives.

```

"""Listing 4: SVM soft margin -- C controls the margin/violation trade, support vectors
tell the story."""
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = make_blobs(n_samples=400, centers=2, cluster_std=1.6, random_state=6)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.4, random_state=6)

print(f"{'C':>8} {'support vecs':>13} {'margin width':>13} {'train acc':>10} {'test
acc':>9}")
for C in [0.01, 1, 100]:
    m = SVC(kernel="linear", C=C).fit(X_tr, y_tr)
    margin = 2 / np.linalg.norm(m.coef_) # geometric margin = 2 / ||w||
    print(f"{'C':>8g} {'m.n_support_.sum():>13} {'margin:>13.3f} "
          f"{'m.score(X_tr, y_tr):>10.3f} {'m.score(X_te, y_te):>9.3f}")

```

Output:

| C    | support vecs | margin width | train acc | test acc |
|------|--------------|--------------|-----------|----------|
| 0.01 | 48           | 3.882        | 0.988     | 0.975    |
| 1    | 10           | 1.347        | 0.992     | 0.969    |
| 100  | 7            | 1.036        | 0.992     | 0.969    |

## Listing 5 — The kernel trick, verified to machine precision

Concentric circles: a linear SVM scores 0.510 — coin-flip, since no hyperplane encloses a disk — while degree-2 polynomial and RBF kernels both reach 0.995. Then the trick itself, with the books balanced explicitly: the degree-2 kernel  $(a \cdot b)^2$  equals the inner product of the explicit feature maps  $\phi(a) \cdot \phi(b)$ , computed without ever materializing  $\phi$ .

```

"""Listing 5: the kernel trick -- circles data, and a kernel computed two ways."""
import numpy as np
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X, y = make_circles(n_samples=500, factor=0.5, noise=0.1, random_state=8)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.4, random_state=8)

for kernel in ["linear", "poly", "rbf"]:
    m = SVC(kernel=kernel, degree=2, C=1).fit(X_tr, y_tr)
    print(f"{'kernel:>7} kernel: test acc = {m.score(X_te, y_te):.3f}")

# The trick itself: K(a,b) = (a.b)^2 equals a dot product in feature space
# phi(a) = (a1^2, a2^2, sqrt(2) a1 a2) -- explicit degree-2 map for 2D input
a, b = X[0], X[1]

```

```
phi = lambda v: np.array([v[0]**2, v[1]**2, np.sqrt(2) * v[0] * v[1]])
lhs = (a @ b) ** 2 # kernel: O(d) work
rhs = phi(a) @ phi(b) # explicit map: O(d^2) features, more for higher degree
print(f"\n(a.b)^2 = {lhs:.6f}    phi(a).phi(b) = {rhs:.6f}    equal: {np.isclose(lhs, rhs)}")
```

Output:

```
linear kernel: test acc = 0.510
poly kernel: test acc = 0.995
rbf kernel: test acc = 0.995

(a.b)^2 = 0.000293    phi(a).phi(b) = 0.000293    equal: True
```

## Listing 6 — Decision tree splitting by hand

A 12-row loan dataset, both impurity criteria implemented from scratch, and exhaustive search over every feature and threshold — exactly what a tree does at every node. Parent entropy is exactly 1 bit (six approvals, six rejections). Both criteria choose the same split, on different scales: the default-history feature, gaining 0.655 bits (entropy) or 0.357 (Gini).

```
"""Listing 6: decision tree splitting from scratch -- information gain and Gini, by hand."""
import numpy as np

# Toy loan dataset: (income_k, has_default_history) -> approved
X = np.array([[25, 1], [35, 1], [45, 0], [20, 0], [65, 0], [80, 0], [30, 1], [90, 0],
              [55, 1], [70, 0], [40, 0], [85, 1]])
y = np.array([0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0])

def entropy(y):
    if len(y) == 0: return 0.0
    p = np.bincount(y, minlength=2) / len(y)
    p = p[p > 0]
    return -(p * np.log2(p)).sum()

def gini(y):
    if len(y) == 0: return 0.0
    p = np.bincount(y, minlength=2) / len(y)
    return 1 - (p ** 2).sum()

def best_split(X, y, impurity):
    base = impurity(y)
    best = (None, None, -1) # feature, threshold, gain
    for j in range(X.shape[1]):
        for t in np.unique(X[:, j]):
            left, right = y[X[:, j] <= t], y[X[:, j] > t]
            if len(left) == 0 or len(right) == 0: continue
            child = (len(left) * impurity(left) + len(right) * impurity(right)) / len(y)
            gain = base - child # information gain (weighted impurity drop)
            if gain > best[2]:
                best = (j, t, gain)
    return best

print(f"parent entropy = {entropy(y):.4f}, parent gini = {gini(y):.4f}")
for name, imp in [("entropy", entropy), ("gini", gini)]:
```

```
j, t, g = best_split(X, y, imp)
feat = ["income", "default_history"][j]
print(f"best split by {name:>7}: {feat} <= {t}    gain = {g:.4f}")
```

Output:

```
parent entropy = 1.0000, parent gini = 0.5000
best split by entropy: default_history <= 0    gain = 0.6549
best split by    gini: default_history <= 0    gain = 0.3571
```

## Listing 7 — Tree overfitting and cost-complexity pruning

With 15% label noise, the unrestricted tree grows 90 leaves and memorizes (train 1.000, test 0.798). Cost-complexity pruning — grow full, prune by cross-validated  $\alpha$  — lands at 11 leaves and test 0.842, beating every fixed depth cap. The lesson: let the tree see everything, then charge rent per leaf.

```
"""Listing 7: tree overfitting and cost-complexity pruning."""
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier

X, y = make_classification(n_samples=1200, n_features=10, n_informative=5,
                          flip_y=0.15, random_state=9) # 15% label noise
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.4, random_state=9)

print(f"{'max_depth':>9} {'leaves':>7} {'train acc':>10} {'test acc':>9}")
for depth in [2, 4, 8, None]:
    t = DecisionTreeClassifier(max_depth=depth, random_state=9).fit(X_tr, y_tr)
    print(f"{'str(depth):>9} {t.get_n_leaves():>7} {t.score(X_tr, y_tr):>10.3f} "
          f"{t.score(X_te, y_te):>9.3f}")

# Cost-complexity pruning: grow full, then prune by CV over alpha
path = DecisionTreeClassifier(random_state=9).cost_complexity_pruning_path(X_tr, y_tr)
best_alpha, best_acc = 0, 0
for a in np.unique(path.ccp_alphas)[:3]:
    acc = cross_val_score(DecisionTreeClassifier(ccp_alpha=a, random_state=9), X_tr,
                          y_tr, cv=5).mean()
    if acc > best_acc: best_alpha, best_acc = a, acc
pruned = DecisionTreeClassifier(ccp_alpha=best_alpha, random_state=9).fit(X_tr, y_tr)
print(f"\npruned (alpha={best_alpha:.5f}): {pruned.get_n_leaves()} leaves, "
      f"train {pruned.score(X_tr, y_tr):.3f}, test {pruned.score(X_te, y_te):.3f}")
```

Output:

| max_depth | leaves | train acc | test acc |
|-----------|--------|-----------|----------|
| 2         | 4      | 0.735     | 0.725    |
| 4         | 15     | 0.847     | 0.810    |
| 8         | 57     | 0.958     | 0.810    |
| None      | 90     | 1.000     | 0.798    |

pruned (alpha=0.00802): 11 leaves, train 0.854, test 0.842

## Listing 8 — Gini vs entropy: measured, not argued

Forty datasets, identical trees except for the criterion. Neither wins systematically (22–18), the mean accuracy difference is under two points with no consistent direction, and fit times are equal. The empirical answer to a classic question: pick either, spend your attention on depth and pruning.

```

"""Listing 8: Gini vs entropy -- does the criterion actually matter? Measured."""
import time
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

rng = np.random.default_rng(13)
same_acc, gini_wins, ent_wins, t_gini, t_ent = 0, 0, 0, 0.0, 0.0
diffs = []
trials = 40
for i in range(trials):
    X, y = make_classification(n_samples=800, n_features=12, n_informative=6,
                              flip_y=0.1, random_state=i)
    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.4, random_state=i)
    t0 = time.perf_counter()
    ag = DecisionTreeClassifier(criterion="gini", max_depth=6, random_state=0)
    ag.fit(X_tr, y_tr).score(X_te, y_te)
    t_gini += time.perf_counter() - t0
    t0 = time.perf_counter()
    ae = DecisionTreeClassifier(criterion="entropy", max_depth=6, random_state=0)
    ae.fit(X_tr, y_tr).score(X_te, y_te)
    t_ent += time.perf_counter() - t0
    diffs.append(abs(ag - ae))
    if abs(ag - ae) < 1e-12: same_acc += 1
    elif ag > ae: gini_wins += 1
    else: ent_wins += 1

print(f"over {trials} datasets: identical acc {same_acc}, gini better {gini_wins},
entropy better {ent_wins}")
print(f"mean |accuracy difference| = {np.mean(diffs):.4f}")
print(f"total fit time: gini {t_gini:.2f}s, entropy {t_ent:.2f}s")

```

Output:

```

over 40 datasets: identical acc 0, gini better 22, entropy better 18
mean |accuracy difference| = 0.0194
total fit time: gini 0.16s, entropy 0.15s

```

## Pitfalls, comparisons and practical tips

The four algorithms, side by side:

|                 | KNN               | Naive Bayes        | SVM (kernel)        | Decision Tree         |
|-----------------|-------------------|--------------------|---------------------|-----------------------|
| Training cost   | $O(1)$            | $O(nd)$ , one pass | $O(n^2)$ – $O(n^3)$ | $O(n \, d \, \log n)$ |
| Prediction cost | $O(nd)$ per query | $O(cd)$            | $O(\#SV \cdot d)$   | $O(\text{depth})$     |

|                         | KNN                     | Naive Bayes                    | SVM (kernel)              | Decision Tree                  |
|-------------------------|-------------------------|--------------------------------|---------------------------|--------------------------------|
| Needs scaling           | yes — critical          | no                             | yes (RBF/poly)            | no                             |
| Native probabilities    | vote fractions (coarse) | yes (miscalibrated)            | no (Platt bolt-on)        | leaf fractions (overconfident) |
| Decision boundary       | local, jagged           | quadratic (GNB) / linear (MNB) | max-margin, kernel-shaped | axis-aligned staircase         |
| Key dial(s)             | k, metric               | $\alpha$ smoothing             | C, $\gamma$ , kernel      | depth / pruning $\alpha$       |
| Curse of dimensionality | fatal                   | tolerant                       | tolerant (margin bounds)  | greedy search dilutes          |
| Interpretability        | by example              | per-feature log-evidence       | low (kernel)              | highest                        |

**When-to-reach-for-which, one line each.** KNN: small data, local structure, or retrieval over embeddings. Naive Bayes: text baselines, tiny data, streaming counts, need for speed. SVM: medium clean datasets, high-d/low-n, when maximum-margin robustness matters. Tree: when the stakeholder must read the model — otherwise its ensemble descendants (Chapter 7) take its place.

#### Classic traps:

- **Forgetting to scale KNN or RBF-SVM features.** Both are distance machines; an ill-scaled feature silently owns every distance (Listing 1's nine-point drop). Trees and NB are immune — knowing *which* models need scaling and why is a favorite screening question.
- **Reading NB probabilities as calibrated.** The independence violation inflates confidence toward 0/1; ranking survives, probabilities don't. Recalibrate (Chapter 10) or use labels only.
- **Skipping smoothing in Multinomial NB.** One unseen word-class pair vetoes the class with  $\log(0)$ . Laplace smoothing is not optional (Listing 3).
- **Tuning C without  $\gamma$  (or vice versa) in RBF SVMs.** They interact — a good C at one  $\gamma$  is terrible at another; grid or Bayesian-search them jointly, on log scales.
- **Believing a deep tree's training accuracy.** 1.000 train / 0.798 test (Listing 7) is the default outcome, not an anomaly; always prune or cap, always by validation.
- **Trusting impurity-based feature importance.** Biased toward high-cardinality and continuous features; an ID column looks maximally informative. Permutation importance (Chapter 11) before conclusions.
- **Fitting kernel SVMs on 500k rows.** The quadratic-plus kernel matrix will not fit and the QP will not finish. Linear SVM (liblinear/SGD) or a different model family.
- **Using k=1 "because it's most flexible".** It is also the maximum-variance member of the family, with an expected error approaching twice the Bayes rate asymptotically — a theory fact interviewers enjoy: 1-NN's asymptotic error is



bounded by  $2 \times$  Bayes error, so even the crudest rule is within a factor of two of optimal given infinite data.

- **One-hot exploding categorical features for trees.** Trees can split subsets natively (implementation permitting); one-hot fragments a categorical's signal across many shallow binary splits and biases against it. Know your library's categorical handling (Chapter 7's LightGBM/CatBoost treat this as a headline feature).

**Threshold and imbalance notes, shared across all four.** Every classifier here yields scores; the 0.5-equivalent default threshold assumes symmetric costs (Chapter 5's logistic discussion applies verbatim). Under heavy imbalance: stratify splits (Chapter 4), weight classes (`class_weight` in sklearn's SVM/tree; priors in NB do it natively), and evaluate with PR-AUC rather than accuracy (Chapter 10).

## Interview questions and answers

### Q1. Why is KNN called a "lazy" and "non-parametric" learner, and what are the consequences of each label?

Lazy: no training computation — the model is the stored dataset, and all work happens at query time. Consequence:  $O(1)$  training but  $O(nd)$  per prediction, the inverse of the usual profile, which disqualifies brute-force KNN from low-latency serving at scale. Non-parametric: no fixed-size parameter vector — model complexity grows with the data (Chapter 4's distinction). Consequence: capacity adapts to any boundary shape given enough data, but memory holds the full training set forever and generalization leans entirely on the smoothness assumption that nearby points share labels.

### Q2. How does $k$ in KNN relate to bias and variance? What happens at $k=1$ and $k=n$ ?

$k$  is a smoothing dial: small  $k$  means small neighborhoods — flexible, jagged boundaries, high variance; large  $k$  means votes pooled over wide regions — smooth boundaries, high bias.  $k=1$  memorizes (train accuracy 1.0 by construction; the boundary is the Voronoi diagram of the training set, tracing every noise point).  $k=n$  predicts the global majority class everywhere, ignoring  $x$  entirely. Listing 1 traces the arc: 1.000/0.892 train/test at  $k=1$ , best test at moderate  $k$ , sagging accuracy by  $k=201$ . Select  $k$  by cross-validation. Bonus theory: 1-NN's asymptotic error is at most twice the Bayes error.

**Q3. Why does feature scaling matter enormously for KNN and RBF-SVMs but not at all for decision trees or Naive Bayes?**

KNN and the RBF kernel are built on distances, and squared differences add across features in raw units — a feature with  $1000\times$  the scale contributes  $10^6\times$  the squared distance, deciding neighborhoods alone (Listing 1: accuracy  $0.887 \rightarrow 0.792$  unscaled). Trees only compare each feature against its own thresholds — any monotone transform leaves the split structure unchanged. NB models each feature independently in its own units; scale is absorbed into the per-feature densities/counts. The pattern worth stating: scaling matters wherever features are *combined* in a common metric (distances, penalties, gradient steps) and not where they are treated per-feature.

**Q4. State the Naive Bayes assumption precisely. It's false in practice — why does the classifier still work?**

Conditional independence: features are independent *given the class*, so the joint likelihood factorizes into per-feature terms. (Not marginal independence — features may correlate through the class.) It survives its own falseness because prediction needs only the argmax over classes: double-counting correlated evidence inflates the winner's posterior but often preserves which class wins. The casualty is calibration — probabilities are pushed toward 0/1 and shouldn't be trusted without recalibration. The crisp phrasing: the decision is robust, the probabilities are not.

**Q5. When would you use Gaussian vs Multinomial vs Bernoulli Naive Bayes?**

By feature type, because the variants differ only in the likelihood model. Gaussian: continuous features (per-class mean/variance per feature). Multinomial: counts — the text default with bag-of-words or TF vectors, where a document's score is a count-weighted sum of word log-probabilities. Bernoulli: binary presence/absence — and unlike Multinomial it also scores *absent* features via the  $(1-p)$  factors, which can win on short documents where a word's absence is informative. Multinomial usually wins on longer text where repetition carries signal.

**Q6. Why is Laplace smoothing necessary in Multinomial NB, and what is it Bayesianly?**

An unseen word-class pair has count 0, hence estimated probability 0, hence log-probability  $-\infty$  — a single novel word vetoes the entire class regardless of all other evidence (Listing 3's 'deadline' example). Add- $\alpha$  smoothing replaces count/total with  $(\text{count} + \alpha) / (\text{total} + \alpha V)$ , guaranteeing every word positive probability. Bayesianly it is MAP estimation under a Dirichlet( $\alpha$ ) prior on the word distribution — pseudo-counts as prior belief, Chapter 1's MLE-vs-MAP story again.  $\alpha$  tunes shrinkage toward uniform: larger  $\alpha$  = more smoothing = more bias, less variance.

**Q7. Why must NB implementations compute in log space?**

The likelihood is a product of many per-feature probabilities, each possibly tiny; a few hundred factors underflow float64 to exactly zero, making all classes tie at 0. Logs turn the product into a sum of manageable negative numbers and turn argmax of products into argmax of sums (log is monotone). Same discipline appears anywhere likelihoods multiply — HMMs, language model scoring (Chapter 20). If actual probabilities are needed, normalize with the log-sum-exp trick, subtracting the max before exponentiating.

**Q8. Define the margin of an SVM and derive why maximizing it means minimizing  $\|w\|^2$ .**

The margin is the distance from the separating hyperplane  $w^T x + b = 0$  to the closest training point. A point's distance to the hyperplane is  $|w^T x + b| / \|w\|$ . Fix the scale ambiguity ( $w, b$  can be multiplied by any constant) by canonicalizing: the closest points satisfy  $|w^T x + b| = 1$ . Then the closest distance is  $1 / \|w\|$  and the full margin between the two supporting hyperplanes is  $2 / \|w\|$ . Maximizing  $2 / \|w\| =$  minimizing  $\|w\| =$  minimizing  $\frac{1}{2} \|w\|^2$  (squared for differentiability, halved for a clean gradient), subject to  $y_i(w^T x_i + b) \geq 1$ . *Interviewers listen for: the canonicalization step — without it "maximize the margin" has no well-defined objective.*

**Q9. What is a support vector, and why is the SVM solution sparse in them?**

A training point with a nonzero dual coefficient — geometrically, one lying on the margin (or violating it, in soft-margin). The optimal  $w$  is a weighted sum of support vectors only; every point comfortably beyond the margin has zero hinge loss and zero gradient pull, so removing it changes nothing. Consequences: prediction cost scales with the number of SVs, not  $n$ ; the SV count is a capacity diagnostic (Listing 4: 48 SVs at  $C=0.01$  vs 7 at  $C=100$ ); and a very high SV fraction signals an overlapping or misspecified problem.

**Q10. Hard margin vs soft margin — what problem does soft margin solve, and what does  $C$  control?**

Hard margin requires perfect separation: it has no solution on non-separable data and is hostage to its single closest point (one mislabeled example relocates the boundary). Soft margin adds slack  $\xi_i \geq 0$  per point — the amount by which it may violate the margin — and charges  $C$  per unit of total slack.  $C$  is the bias-variance dial: large  $C$  approximates hard margin (narrow, contorted, high variance), small  $C$  buys width by tolerating violations (smooth, high bias). Note  $C$  multiplies the loss term, so it behaves like  $1/\lambda$  — the inverse of ridge's  $\lambda$ , a sign-convention trap shared with sklearn's LogisticRegression.

**Q11. Explain the kernel trick precisely — what property of the SVM makes it possible?**

In the dual, both training and prediction touch data *only through inner products*  $x_i^T x_j$ . Replace every inner product with  $K(a,b) = \phi(a)^T \phi(b)$  for some feature map  $\phi$ , and the algorithm runs identically — fitting a max-margin hyperplane in  $\phi$ -space without ever computing  $\phi$ . The payoff:  $K$  is often far cheaper than  $\phi$  (Listing 5:  $(a^T b)^2$  in  $O(d)$  equals the inner product of an  $O(d^2)$ -dimensional explicit map, verified to machine precision); for RBF,  $\phi$ -space is infinite-dimensional yet  $K$  costs  $O(d)$ . Any symmetric positive semi-definite function is valid (Mercer). Applies to any algorithm expressible in inner products — kernelized ridge, PCA, and friends.

**Q12. What does the RBF kernel's  $\gamma$  control? Describe overfitting with large  $\gamma$ .**

$K(a,b) = \exp(-\gamma\|a-b\|^2)$ :  $\gamma$  is inverse bandwidth — the reach of each support vector's influence. Small  $\gamma$ : broad influence, near-linear smooth boundaries, high bias. Large  $\gamma$ : influence shrinks to tiny neighborhoods, the boundary shrink-wraps individual training points — islands around each example, training accuracy  $\sim 1$ , terrible generalization; in the  $\gamma \rightarrow \infty$  limit behavior approaches 1-NN memorization.  $\gamma$  and  $C$  interact (both push flexibility), so tune jointly on a log grid, with scaled features — the kernel is a distance and inherits distance's scale-sensitivity.

**Q13. Compare hinge loss and cross-entropy as classification losses.**

Hinge:  $\max(0, 1 - y \cdot z)$  — exactly zero beyond the margin, so correctly-and-confidently classified points exert no pull; produces sparse solutions (support vectors) and stops caring once clearance is achieved. Linear growth for violators makes it more outlier-tolerant than squared losses. Not differentiable at the kink (subgradients handle it); no probabilistic reading. Cross-entropy: never exactly zero — every point keeps pulling, weights keep growing without regularization on separable data (Chapter 5); smooth, convex, and is the Bernoulli MLE, so it yields calibrated probabilities. Rule of thumb: hinge when you want a decision with margin sparsity, cross-entropy when you want probabilities.

**Q14. Why do kernel SVMs struggle at large  $n$ , and what are the practical outs?**

The kernel matrix is  $n \times n$  — a million points means  $10^{12}$  entries — and QP solvers run  $O(n^2)$ – $O(n^3)$ ; prediction also scales with the support-vector count, which grows with  $n$  on noisy data. Outs: linear SVMs via liblinear or SGD with hinge loss (text and other high-d sparse data is often near-separable, making the kernel unnecessary); kernel approximations mapping to explicit low-dimensional features (random Fourier features, Nyström) then fitting linear; subsampling; or conceding the regime to boosted trees / neural nets. Knowing the crossover ( $\sim 10^4$ – $10^5$  points) is the practical marker.

**Q15. Walk through how a decision tree chooses a split, including the exact quantity optimized.**

At each node, for every feature and every candidate threshold (midpoints between sorted unique values for continuous features): partition the node's examples, compute the impurity of each child (Gini or entropy), take the size-weighted average, and subtract from the parent's impurity — the information gain. Choose the (feature, threshold) with maximum gain; recurse. Listing 6 does it by hand: parent entropy 1.0 bit, best split gains 0.655 bits. Greedy and local — each split maximizes immediate purification with no lookahead, which is why XOR-structured signals defeat single trees.

**Q16. Gini vs entropy — differences in theory and practice?**

Theory: entropy =  $-\sum p \log_2 p$  (information-theoretic, from Chapter 1); Gini =  $1 - \sum p^2$  (probability of misclassifying by the node's own label distribution). Both zero when pure, maximal when uniform; entropy penalizes near-purity slightly differently and is marginally more expensive (a log per class). Practice: they choose the same splits the vast majority of the time and neither wins systematically — measured across 40 datasets (Listing 8): 22–18 with mean accuracy difference 0.019 and identical fit time. Strong answer: name the formulas, then say the criterion is a rounding error next to depth control and pruning — and know that *gain ratio* (C4.5) exists to fix information gain's bias toward high-cardinality features.

**Q17. Why do unconstrained decision trees overfit, and how do pre-pruning and post-pruning differ?**

Grown to purity, a tree can dedicate a leaf to every hard example — with label noise, that means memorizing noise (Listing 7: 90 leaves, 1.000 train, 0.798 test on 15%-noise data). Pre-pruning stops growth early via caps: `max_depth`, `min_samples_leaf`, `min_impurity_decrease` — cheap, but can stop before a split that would have enabled a good deeper one (the greedy horizon problem). Post-pruning grows the full tree, then collapses subtrees that don't pay for their complexity: cost-complexity pruning penalizes accuracy –  $\alpha \cdot (\# \text{leaves})$ , sweeping  $\alpha$  by cross-validation (Listing 7: 11 leaves, 0.842 test — best of all variants). Post-pruning sees the whole tree before judging, at the cost of growing it first.

**Q18. Why are single decision trees said to have high variance, and what exploits that property?**

The greedy structure compounds instability: a small data perturbation flips the top split, which changes every descendant's data, which changes their splits — entire subtrees reorganize. Deep trees are therefore low-bias, high-variance estimators whose predictions vary wildly across resamples. Bagging (Chapter 7) exploits exactly this: averaging many high-variance, low-bias trees trained on bootstrap resamples cancels the variance while keeping the low bias — random forests are the institutionalized fix, adding feature subsampling to decorrelate the trees further. The interview arc worth telling: the tree's flaw is the forest's fuel.

**Q19. A tree's impurity-based feature importance ranks a user-ID-like column first. Diagnose.**

Cardinality bias: a feature with many unique values offers many candidate thresholds, so it can carve data into small pure groups by chance — an ID column can achieve perfect training splits with zero generalizable signal. Impurity-based importance sums these fake gains. Fixes: drop identifier-like columns (also a leakage check — Chapter 4), use permutation importance on held-out data (Chapter 11), or gain-ratio-style corrections. Follow-up worth volunteering: the same bias inflates continuous features relative to low-cardinality categoricals.

**Q20. Your KNN model is slow at prediction time in production. Options?**

Ordered by invasiveness: reduce  $n$  — prototype selection/condensing, or cluster centroids as the reference set; reduce  $d$  — PCA or feature selection (also helps accuracy via the curse); exact tree indexes — KD-tree/ball tree, effective only up to a few dozen dimensions; approximate nearest neighbors — HNSW graphs, IVF/product quantization (the vector-database stack, Chapter 24), trading tiny recall loss for orders-of-magnitude speedups; or switch families — fit a parametric model (logistic, tree ensemble) if latency dominates. Naming ANN indexes as the industrial answer signals current practice.

**Q21. Concentric-circles data: which of the four classifiers work out of the box, and why?**

KNN: yes — purely local voting, shape-agnostic. Kernel SVM (RBF or degree-2 poly): yes — the boundary is linear in feature space, circular in input space (Listing 5: 0.995). Gaussian NB: yes, by luck of the geometry — per-class Gaussians with different variances yield a quadratic boundary, which can be an ellipse. Decision tree: yes but ugly — axis-aligned staircase approximating a circle, needing depth. Linear SVM / logistic regression: no (0.510 in Listing 5) — no hyperplane encloses a disk — unless you engineer features (add  $x_1^2$ ,  $x_2^2$ , and the problem becomes linearly separable). The question tests whether you can map model class to boundary geometry.

**Q22. Why does Naive Bayes often beat more expressive models on small text datasets?**

Bias-variance at high dimension: text has  $d$  in the tens of thousands and small  $n$ . NB estimates only  $d$  univariate distributions per class — its errors are dominated by (stable, predictable) bias. An expressive model estimating interactions has variance that small  $n$  cannot support. NB's strong-but-wrong independence assumption acts as extreme regularization, and with argmax-only decisions its miscalibration doesn't hurt labels. Empirically NB also benefits from redundant evidence in text: many weakly informative words vote, and vote-counting is robust. As  $n$  grows the ranking flips — linear SVMs and logistic regression overtake it, then neural approaches.

**Q23. How do you get probabilities out of an SVM, and what's the caveat?**

SVMs output margins (signed distances), not probabilities — hinge loss has no likelihood interpretation. Platt scaling fits a one-dimensional logistic regression  $\sigma(a \cdot z + b)$  mapping margin  $z$  to probability, on held-out data (sklearn's `probability=True` does this via internal CV). Caveats: it adds an expensive extra fit; probabilities can be inconsistent with the raw decision function near the boundary (predict vs `predict_proba` can disagree); and calibration quality depends on the held-out fit — verify with reliability curves (Chapter 10). Isotonic regression is the nonparametric alternative given enough data.



**Q24. Two spam features are near-duplicates ("free" and "FREE" as separate tokens). What happens to NB, and to logistic regression, and why the difference?**

NB double-counts: the independence assumption treats the duplicate evidence as fresh, effectively squaring that feature's likelihood contribution — confidence inflates, and with enough duplication the decision itself can tip. Logistic regression, trained discriminatively, sees the redundancy and splits the weight between the copies (Chapter 5's collinearity: coefficient split arbitrary, *sum* — and hence predictions — well-determined). The general principle: generative models estimating  $p(x|y)$  per feature can't notice inter-feature redundancy; discriminative models optimizing  $p(y|x)$  automatically discount it. That's also a lens on why logistic regression usually edges out NB with sufficient data.

**Q25. Sketch KNN, SVM-RBF, and decision-tree decision boundaries on the same 2-D dataset. What do the shapes reveal?**

KNN: locally jagged, following data density; jaggedness decreasing with  $k$ . SVM-RBF: smooth curved contours with a margin corridor, complexity set by  $\gamma$  and  $C$ . Tree: rectangles — axis-parallel line segments meeting at right angles. The shapes are the models' inductive biases made visible: KNN assumes local smoothness, the RBF SVM assumes smooth boundaries with maximal clearance, the tree assumes axis-aligned, hierarchical structure. Diagonal or rotated structure embarrasses the tree; enclosed regions embarrass linear models; fine local texture with small  $n$  embarrasses everything but KNN (which then overfits it). *Interviewers listen for: connecting boundary geometry to inductive bias, not just describing pictures.*

**Q26. You have 1,000 labeled emails and need a working spam filter today, with per-decision explanations. Pick a model and defend it.**

Multinomial NB with Laplace smoothing on bag-of-words. Defense: trains in milliseconds on 1,000 examples; small-n/high-d is NB's winning regime (Q22); explanations are native — each word contributes an additive log-evidence term, so "flagged because: 'prize' (+2.1), 'free' (+1.8)" falls out of the model; and it updates online as new labels arrive by incrementing counts. State the caveats to show judgment: probabilities need recalibration before being shown as confidence scores, and a linear SVM or logistic regression should take over as labels accumulate. *Interviewers listen for: matching constraints (time, data size, explainability) to model properties rather than reaching for the fanciest option.*

**Q27. Implement the decision rule of a 3-NN classifier in one line of NumPy, given distance matrix D (queries × train) and labels y.**

For binary 0/1 labels: `preds = (y[np.argsort(D, axis=1)[:3]].mean(axis=1) >= 0.5).astype(int)` — `argsort` each query's distances, take the three nearest indices, look up labels, majority = mean  $\geq 0.5$ . Points to volunteer: `argsort` is  $O(n \log n)$  per query while `np.argpartition(D, 3, axis=1)` is  $O(n)$  and the right call at scale; ties are impossible with odd  $k$  in binary problems; and for multiclass replace the mean with a `bincount-argmax` per row.

**Q28. For each of the four algorithms, name its single most important hyperparameter and what happens at each extreme.**

KNN —  $k$ :  $k=1$  memorizes (max variance),  $k=n$  predicts the majority class (max bias). NB — smoothing  $\alpha$ :  $\alpha \rightarrow 0$  lets zero counts veto classes (brittle),  $\alpha \rightarrow \infty$  flattens likelihoods toward uniform so only priors matter. SVM —  $C$  (with  $\gamma$  close behind for RBF):  $C \rightarrow \infty$  is hard margin, contorted by every point;  $C \rightarrow 0$  ignores the data for width, collapsing toward the majority side. Tree — depth/pruning  $\alpha$ : unlimited depth memorizes (Listing 7), depth 1 is a decision stump. The unifying observation that makes the answer land: every one of these is the same bias-variance dial from Chapter 4, wearing different notation.